

TDD Report

Geng Yuan, Mingjia Zou

Task1: Answer the following questions in your report:

1. Screenshot of test coverage HTML report.

Execute the code in the corresponding directory:

```
pytest --cov=Task1_Rover --cov=Cipher --cov-branch --cov-report=html
```

Coverage report: 100%

Files Functions Classes

coverage.py v7.13.3, created at 2026-02-14 12:54 +0100

File ▼	function	Statements				Branches			Total
		coverage	statements	missing	excluded	coverage	branches	partial	
Task1_Rover.py	rovar.__init__	100%	2	0	0	100%	0	0	100%
Task1_Rover.py	rovar.enrove	100%	10	0	0	100%	8	0	100%
Task1_Rover.py	rovar.derove	100%	9	0	0	100%	6	0	100%
Task1_Rover.py	(no function)	100%	4	0	0	100%	0	0	100%
Cipher.py	pattern_cipher	100%	16	0	0	100%	8	0	100%
Cipher.py	(no function)	100%	1	0	0	100%	0	0	100%
Total		100%	42	0	0	100%	22	0	100%

2. What Types of coverage are measured?

1) Statement Coverage

Statement coverage measures the percentage of executable code statements that have been run at least once during your tests.

2) Branch Coverage

Branch coverage measures whether every possible path through a decision point has been taken.

3. What was the Code coverage percentage achieved?

100%

4. Describe the bug found in the code?

1) The first bug:

```
class rovar:
    def __init__(self) -> None:
        #Test with all lowercase consonants: (b, c, d, f, g, h, j, k, l, m, n, p, q, r, s, t, v, w, x, z)
        #self._LOWER_CONSTANTS = "bcd fhjklmnpqrstvwxyz"
        self._LOWER_CONSTANTS = "bcd fghjklmnpqrstvwxyz" # add 'g'

        #self._UPPER_CONSTANTS = "BCFGHJ KLMNPQRST VWXZ"
        self._UPPER_CONSTANTS = "BCFDGHJ KLMNPQRST VWXZ" # add 'D'
```

Missing Lowercase Consonant 'g' and Uppercase Consonant 'D'.

2) The Second bug:

```

for c in self._UPPER_CONSTANTS:
    # find = c+'O'+c # should be lowercase
    find = c+'o'+c
    rov = rov.replace(find, c)

return rov

```

In the derove function, the code originally tried to find a pattern using an uppercase 'O', but it should have used a lowercase 'o' to match the encryption rules.

5. Which test case(s) revealed the bug? (Copy-paste test case or screenshot)

```

# Example test case to check lower case rover
def test_enrove_small(self):

    expected = ''.join([c + 'o' + c for c in self.lower_consonants])
    self.assertEqual(self.rv.enrove(self.lower_consonants), expected)

```

```

AssertionError: 'bobcododfoghohjokloImomnonpopqqrorsostotvovwowxoz' != 'bobcododfoghohjokloImomnonpopqqrorsostotvovwowxoz'
- bobcododfoghohjokloImomnonpopqqrorsostotvovwowxoz
+ bobcododfoghohjokloImomnonpopqqrorsostotvovwowxoz
?      ++

```

```

# Test case to check upper case rover
def test_enrove_upper(self):
    expected = ''.join([c + 'o' + c for c in self.upper_consonants])
    self.assertEqual(self.rv.enrove(self.upper_consonants), expected)

```

```

AssertionError: 'BoBCoCFoFDGoGHoHJoJKoKLoLMoMNoNPoPQoQRoRSoSToTVoVWoWxOxZoZ' != 'BoBCoCFoFDGoGHoHJoJKoKLoLMoMNoNPoPQoQRoRSoSToTVoVWoWxXZoZ'
- BoBCoCFoFDGoGHoHJoJKoKLoLMoMNoNPoPQoQRoRSoSToTVoVWoWxOxZoZ
+ BoBCoCFoFDGoGHoHJoJKoKLoLMoMNoNPoPQoQRoRSoSToTVoVWoWxXZoZ
?      ++

```

```

def test_derove_uppercase_consonants_all(self):
    encoded = self.rv.enrove(self.upper_consonants)
    self.assertEqual(self.rv.derove(encoded), self.upper_consonants)

```

```

AssertionError: 'BoBCoCFoFDGoGHoHJoJKoKLoLMoMNoNPoPQoQRoRSoSToTVoVWoWxOxZoZ' != 'BCFDGHJKLMNPQRSTVWXZ'
- BoBCoCFoFDGoGHoHJoJKoKLoLMoMNoNPoPQoQRoRSoSToTVoVWoWxOxZoZ
+ BCFDGHJKLMNPQRSTVWXZ

```

6. How did you fix the bug?

As shown in the pictures of question 4.

Task2: Answer the following questions in your report:

1. You should include a table mapping each test case to its equivalence class

(ID)	Equivalence Class	Test Method	Input x	Expected	Rule
1	Empty/None	test_none_input / test_empty_string	None, ""	None, ""	If the input is empty ("") or None, it is returned unchanged.
2	Non-alphabetic	test_numbers_unchanged	"123", "\$3"	"123", "\$3"	- Only alphabetic words are transformed. - Special characters/punctuation remain unchanged.
3	Palindrome	test_palindrome_lowercase	"level", "civic"	"level", "civic"	If a word is a palindrome (reads the same forward and backward, case-insensitive), leave it unchanged.
4	Repeated, Not Palindrome	test_repeated_letters_reverse	"hello", "letter"	"olleh", "rettel"	If the word contains any repeated letters and is not a palindrome, reverse the word.
5	All Unique Letters	test_unique_letters_rotate	"cat", "Python"	"atc", "ythonP"	- If all letters are unique, rotate the word left by one character. - Original casing is preserved.
6	Rule Priority	test_palindrome_priority_over _repeated	"noon"	"noon"	Some words may match multiple rules. Priority determines which rule is applied: Palindrome > Repeated Letter > Unique Letter
7	Single-Letter	test_single_letter	"a"	"a"	Rule 3
8	Two-Letter	test_two_letter	"ab", "bb"	"ba", "bb"	Rule 5 / Rule 3

2. Screenshot of test coverage HTML report.

Coverage report: 100%

Files

Functions

Classes

coverage.py v7.13.3, created at 2026-02-14 12:54 +0100

File ▼	function	Statements				Branches			Total
		coverage	statements	missing	excluded	coverage	branches	partial	
Task1_Rover.py	rovar.__init__	100%	2	0	0	100%	0	0	100%
Task1_Rover.py	rovar.enrove	100%	10	0	0	100%	8	0	100%
Task1_Rover.py	rovar.derove	100%	9	0	0	100%	6	0	100%
Task1_Rover.py	(no function)	100%	4	0	0	100%	0	0	100%
Cipher.py	pattern_cipher	100%	16	0	0	100%	8	0	100%
Cipher.py	(no function)	100%	1	0	0	100%	0	0	100%
Total		100%	42	0	0	100%	22	0	100%

3. What Types of test coverages are measured?

- Statement Coverage.
- Branch Coverage.

4. What was the Code coverage percentage achieved?

100% for Statements and 100% for Branches.